Lab # 10
Comprehensive Study of PL/SQL: Blocks, Data Types, Conditional Logic, Loops, Procedures, and Functions

National University of computer and emerging sciences

Database Systems Lab (CL-2005)

Semester: Spring 2026
Course Instructors: Mr. Abdullah Shaikh, Mr. Jahanzaib, Mr. Muhammad Khalid, Mr. Osama Tahir, Mr. Sheeraz Iqbal, Ms. Hijaab Sikander, Ms. Kinza Mushtaq, Ms. Ramsha Jat, Ms. Sadaf Zehra, Ms. Saira Ayub, Ms. Ramsha Iqbal

LAB # 10

Comprehensive Study of PL/SQL: Blocks, Data Types, Conditional Logic, Loops, Procedures, and Functions

Objectives:

- To understand the block structure of PL/SQL
- To learn different data types and variable declarations
- To implement conditional statements for decision making
- To use loops for iterative processing
- To understand and create stored procedures
- To learn parameter passing techniques (IN, OUT, IN OUT) in procedures

PL/SQL

PL/SQL (Procedural Language/SQL) is an extension of SQL that combines the data manipulation capabilities of SQL with the procedural features of programming languages. It is a portable, high-performance, transaction-processing language.

PL/SQL provides a built-in, interpreted, and operating system–independent programming environment. It is tightly integrated with SQL and offers extensive error checking, numerous data types, and a variety of programming structures.

It supports structured programming through functions and procedures, as well as object-oriented programming concepts. Additionally, PL/SQL is widely used for developing web applications and server-side programs.

Lab # 10
Comprehensive Study of PL/SQL: Blocks, Data Types, Conditional Logic, Loops, Procedures, and Functions

Block Structure

A PL/SQL program is divided into blocks. The basic structure of a PL/SQL block is as follows:

```
DECLARE
   -- Declarations section
BEGIN
   -- Executable statements
EXCEPTION
   -- Exception handling section
END;
```

Note:

Always include the following command at the top of your script to display output:

```
SET SERVEROUTPUT ON;
```

Example

```
SET SERVEROUTPUT ON;

DECLARE
   Sec_Name VARCHAR2(20) := 'Sec-E';
   Course_Name VARCHAR2(20) := 'Database Systems Lab';
BEGIN
   DBMS_OUTPUT.PUT_LINE('This is: ' || Sec_Name ||
                        ' and the course is ' || Course_Name);
END;
/
```

Common Delimiters and Operators

Delimiter	Description	Delimiter	Description
+, -, *, /	Arithmetic operators (addition, subtraction, multiplication, division)	:	Host variable indicator
%	Attribute indicator	,	Item separator
'	Character string delimiter	"	Quoted identifier delimiter
.	Component selector	=	Relational operator
( )	Expression or list delimiter	@	Remote access indicator
||	Concatenation operator	;	Statement terminator
****	Exponentiation operator	:=	Assignment operator
<<, >>	Label delimiters	=>	Association operator
/*, */	Multi-line comment delimiters	<, >, <=, >=	Relational operators
--	Single-line comment	<>, !=, ~=, ^=	Not equal operators
..	Range operator		

Lab # 10
Comprehensive Study of PL/SQL: Blocks, Data Types, Conditional Logic, Loops, Procedures, and Functions

Variables & Types

PL/SQL allows declaration of variables with different data types such as INTEGER, NUMBER, VARCHAR2, etc.

This example demonstrates declaration of variables, performing arithmetic operations, and displaying output.

Example

```
SET SERVEROUTPUT ON;

DECLARE
   a INTEGER := 10;
   b INTEGER := 20;
   c INTEGER;
   f NUMBER;
BEGIN
   c := a + b;
   DBMS_OUTPUT.PUT_LINE('Value of c: ' || c);

   f := 70.0 / 3.0;
   DBMS_OUTPUT.PUT_LINE('Value of f: ' || f);
END;
/
```

Variable Scope (Global vs Local Variables)

This example demonstrates how inner (local) variables override outer (global) variables.

```
SET SERVEROUTPUT ON;

DECLARE
   -- Global variables
   num1 NUMBER := 95;
   num2 NUMBER := 85;
BEGIN
   DBMS_OUTPUT.PUT_LINE('Outer Variable num1: ' || num1);
   DBMS_OUTPUT.PUT_LINE('Outer Variable num2: ' || num2);

   DECLARE
      -- Local variables
      num1 NUMBER := 195;
      num2 NUMBER := 185;
   BEGIN
      DBMS_OUTPUT.PUT_LINE('Inner Variable num1: ' || num1);
      DBMS_OUTPUT.PUT_LINE('Inner Variable num2: ' || num2);
   END;

END;
/
```

Data Retrieval Using SELECT INTO

This example demonstrates how to fetch data from database tables using %TYPE and SELECT INTO.

```
SET SERVEROUTPUT ON;

DECLARE
   e_id     employees.EMPLOYEE_ID%TYPE;
   e_name  employees.FIRST_NAME%TYPE;
   e_lname employees.LAST_NAME%TYPE;
   d_name departments.DEPARTMENT_NAME%TYPE;
BEGIN
   SELECT e.EMPLOYEE_ID, e.FIRST_NAME, e.LAST_NAME, d.DEPARTMENT_NAME
   INTO e_id, e_name, e_lname, d_name
   FROM employees e
   INNER JOIN departments d
   ON e.DEPARTMENT_ID = d.DEPARTMENT_ID
   WHERE e.EMPLOYEE_ID = 100;

   DBMS_OUTPUT.PUT_LINE('EMPLOYEE ID: ' || e_id);
   DBMS_OUTPUT.PUT_LINE('EMPLOYEE First Name: ' || e_name);
   DBMS_OUTPUT.PUT_LINE('EMPLOYEE Last Name: ' || e_lname);
   DBMS_OUTPUT.PUT_LINE('DEPARTMENT Name: ' || d_name);
END;
/
```

Lab # 10
Comprehensive Study of PL/SQL: Blocks, Data Types, Conditional Logic, Loops, Procedures, and Functions

Conditional Logic in PL/SQL

IF–THEN Statement

This example updates salary if it meets a specific condition.

```
SET SERVEROUTPUT ON;

DECLARE
   e_id  employees.EMPLOYEE_ID%TYPE := 100;
   e_sal employees.SALARY%TYPE;
BEGIN
   SELECT salary INTO e_sal
   FROM employees
   WHERE EMPLOYEE_ID = e_id;

   IF (e_sal >= 5000) THEN
      UPDATE employees
      SET salary = e_sal + 1000
      WHERE EMPLOYEE_ID = e_id;

      DBMS_OUTPUT.PUT_LINE('Salary updated');
   END IF;
END;
/
```

IF–THEN–ELSE Statement

This example checks whether a record exists before inserting.

```
SET SERVEROUTPUT ON;

DECLARE
   e_id  employees.EMPLOYEE_ID%TYPE := 100;
   e_sal employees.SALARY%TYPE;
BEGIN
   SELECT salary INTO e_sal
   FROM employees
   WHERE EMPLOYEE_ID = e_id;

   IF (e_sal <= 15000) THEN
      UPDATE employees SET salary = e_sal + 300 WHERE EMPLOYEE_ID = e_id;

   ELSIF (e_sal <= 20000) THEN
      UPDATE employees SET salary = e_sal + 200 WHERE EMPLOYEE_ID = e_id;

   ELSIF (e_sal <= 25000) THEN
      UPDATE employees SET salary = e_sal + 100 WHERE EMPLOYEE_ID = e_id;

   ELSE
      UPDATE employees SET salary = e_sal + 400 WHERE EMPLOYEE_ID = e_id;
   END IF;

   DBMS_OUTPUT.PUT_LINE('Salary updated: ' || e_sal);
END;
/
```

CASE Statement

Simple CASE based on department ID.

```
SET SERVEROUTPUT ON;

DECLARE
   e_id  employees.EMPLOYEE_ID%TYPE := 100;
   e_sal employees.SALARY%TYPE;
   e_did employees.DEPARTMENT_ID%TYPE;
BEGIN
   SELECT salary, department_id
   INTO e_sal, e_did
   FROM employees
   WHERE EMPLOYEE_ID = e_id;

   CASE
      WHEN e_did = 80 THEN
         UPDATE employees SET salary = e_sal + 100 WHERE EMPLOYEE_ID = e_id;

      WHEN e_did = 50 THEN
         UPDATE employees SET salary = e_sal + 200 WHERE EMPLOYEE_ID = e_id;

      WHEN e_did = 40 THEN
         UPDATE employees SET salary = e_sal + 300 WHERE EMPLOYEE_ID = e_id;

      ELSE
         DBMS_OUTPUT.PUT_LINE('No such record');
   END CASE;

   DBMS_OUTPUT.PUT_LINE('Salary updated: ' || e_sal);
END;
/
```

Nested IF–THEN–ELSE

This example demonstrates nested conditions based on department and salary.

```
SET SERVEROUTPUT ON;

DECLARE
   e_id  employees.EMPLOYEE_ID%TYPE := 100;
   e_sal employees.SALARY%TYPE;
   e_did employees.DEPARTMENT_ID%TYPE;
   e_com employees.COMMISSION_PCT%TYPE;
BEGIN
   SELECT salary, department_id, commission_pct
   INTO e_sal, e_did, e_com
   FROM employees
   WHERE EMPLOYEE_ID = e_id;

   IF (e_did = 90) THEN
      IF (e_sal BETWEEN 20000 AND 25000) THEN
         UPDATE employees SET salary = e_sal * (1 + e_com) WHERE EMPLOYEE_ID = e_id;

      ELSIF (e_sal BETWEEN 15000 AND 20000) THEN
         UPDATE employees SET salary = (e_sal + 20) * (1 + e_com) WHERE EMPLOYEE_ID = e_id;
      END IF;
   END IF;

   IF (e_did = 40) THEN
      IF (e_sal BETWEEN 10000 AND 15000) THEN
         UPDATE employees SET salary = e_sal * (1 + e_com) WHERE EMPLOYEE_ID = e_id;

      ELSIF (e_sal BETWEEN 5000 AND 10000) THEN
         UPDATE employees SET salary = (e_sal + 20) * (1 + e_com) WHERE EMPLOYEE_ID = e_id;
      END IF;
   END IF;

   DBMS_OUTPUT.PUT_LINE('Salary processing completed.');
END;
/
```

FOR LOOP

A **FOR LOOP** in PL/SQL is used to iterate over a range of values or a result set returned by a query. It automatically handles loop initialization, condition checking, and increment.

Example: Loop Through Employee Records

This example retrieves employees from **Department 90** and displays their salaries.

```
SET SERVEROUTPUT ON;

DECLARE
BEGIN
   FOR c IN (
      SELECT EMPLOYEE_ID, FIRST_NAME, SALARY
      FROM employees
      WHERE DEPARTMENT_ID = 90
   )
   LOOP
      DBMS_OUTPUT.PUT_LINE(
         'Salary for the employee ' || c.FIRST_NAME ||
         ' is: ' || c.SALARY
      );
   END LOOP;
END;
/
```

Explanation:

- The FOR loop iterates over each row returned by the SELECT query
- c acts as a record variable holding each row
- Fields are accessed using dot notation (e.g., c.FIRST_NAME)
- DBMS_OUTPUT.PUT_LINE is used to display results

Lab # 10
Comprehensive Study of PL/SQL: Blocks, Data Types, Conditional Logic, Loops, Procedures, and Functions

Stored Procedures in PL/SQL

A **stored procedure** is a named PL/SQL block that performs a specific task or a set of tasks. It can contain SQL statements, accept parameters, and execute complex business logic.

Stored procedures are similar to functions or methods in programming languages, but they are stored and executed within the database.

Benefits of Stored Procedures

- **Reusability:**
  A procedure can be created once and reused multiple times by simply calling it.
- **Easy Maintenance:**
  Any change in logic needs to be made only once in the procedure instead of updating multiple SQL statements.
- **Improved Performance:**
  Since procedures are precompiled, execution is faster.

Example: Creating and Executing a Stored Procedure

This procedure inserts a new record into the LOCATIONS table.

```
SET SERVEROUTPUT ON;

CREATE OR REPLACE PROCEDURE Insert_Data (
    STREET_ADDRESS IN VARCHAR2,
    POSTAL_CODE   IN VARCHAR2 DEFAULT NULL,
    CITY          IN VARCHAR2,
    STATE_PROVINCE IN VARCHAR2,
    COUNTRY_ID     IN CHAR
)
IS
    total_record NUMBER;
    location_id  NUMBER;
BEGIN
    -- Generate new LOCATION_ID
    SELECT COUNT(LOCATION_ID) + 1
    INTO location_id
    FROM LOCATIONS;

    total_record := location_id;

    -- Insert new record
    INSERT INTO LOCATIONS (
        LOCATION_ID,
        STREET_ADDRESS,
        POSTAL_CODE,
        CITY,
        STATE_PROVINCE,
        COUNTRY_ID
    )
    VALUES (
        location_id,
        STREET_ADDRESS,
        POSTAL_CODE,
        CITY,
        STATE_PROVINCE,
        COUNTRY_ID
    );

    DBMS_OUTPUT.PUT_LINE('New record inserted with ID: ' || location_id);
    DBMS_OUTPUT.PUT_LINE('Total number of records: ' || total_record);
END;
/
```

Executing the Procedure

```
EXEC Insert_Data('DHA', '1234', 'KARACHI', 'SINDH', 'PK');
```

Passing Parameters in Stored Procedures

Parameters allow values to be passed between the calling program and the stored procedure. PL/SQL supports three types (modes) of parameters:

Types of Parameters

1. IN Parameter

- Default mode
- Used to pass values **into** the procedure
- Cannot be modified inside the procedure

2. OUT Parameter

- Used to return values **from** the procedure
- Value is assigned inside the procedure

3. IN OUT Parameter

- Used to pass a value into the procedure and return a modified value

Example 1: IN Parameter

```
SET SERVEROUTPUT ON;

CREATE OR REPLACE PROCEDURE Show_Employee (
    e_id IN NUMBER
)
IS
    e_name employees.FIRST_NAME%TYPE;
BEGIN
    SELECT FIRST_NAME INTO e_name
    FROM employees
    WHERE EMPLOYEE_ID = e_id;

    DBMS_OUTPUT.PUT_LINE('Employee Name: ' || e_name);
END;
/

-- Execution
EXEC Show_Employee(100);
```

Example 2: OUT Parameter

```
SET SERVEROUTPUT ON;

CREATE OR REPLACE PROCEDURE Get_Salary (
    e_id IN NUMBER,
    e_sal OUT NUMBER
)
IS
BEGIN
    SELECT SALARY INTO e_sal
    FROM employees
    WHERE EMPLOYEE_ID = e_id;
END;
/

-- Execution
DECLARE
    salary NUMBER;
BEGIN
    Get_Salary(100, salary);
    DBMS_OUTPUT.PUT_LINE('Salary: ' || salary);
END;
/
```

Example 3: IN OUT Parameter

```
SET SERVEROUTPUT ON;

CREATE OR REPLACE PROCEDURE Update_Value (
num IN OUT NUMBER
)
IS
BEGIN
num := num + 10;
END;
/

                Execution
DECLARE
value NUMBER := 50;
BEGIN
Update_Value(value);
DBMS_OUTPUT.PUT_LINE('Updated Value: ' || value);
END;
/
```

THE END!